ProveIt research note

K215: Checked Components for Shorter Correctness Proofs

Guarded control frames and extensional source rewriting in Lean and Rocq

Prepared for Vladimir Reshetnikov

22 October 2026

Abstract

The accepted 215-state binary Turing machine already has substantial proof simplifications. The next useful step is to prove reusable statements about unchanged context, then instantiate them at repeated occurrences. This note turns two existing research proposals into checked components.

In Lean 4.32.0, we prove a guarded append-left frame rule for the maintained symbolic executor, check all 35 proposed control templates, and derive complete whole-tape and source-action contracts for every template. An independent audit binds those templates to 1,422 control occurrences of the exact current source. The proposed mixed proof has 4,789 expensive replay obligations instead of 6,176, while retaining all 6,176 source-outcome bindings. This is an obligation count, not a measured endpoint speedup or a completed replacement endpoint. An independent Rocq port checks the same 35 local replays and their framed reachability; its Home and source-action adapters remain integration work.

For the Rocq source proof, we introduce pointwise register transport, avoiding functional extensionality, and prove that reversing exactly shared nodes 188 and 190 of `dagPerm` preserves all structured exits and iterated halting. These components are checked with Coq 8.18.0 and recursive `coqchk`; the production Rocq 9.1.0 endpoint is a separate integration target.

The report gives mathematical proofs, exact integration obligations, failure examples for unsafe simplifications, provenance, and further research questions. It does not change the literal machine, claim an unconditional consistency result, or claim a fresh build of either full endpoint.

Repository snapshot

`cbf94cb552a3b49509e0eee80e7455d4eb5974c6`

Primary deliverable: checked reusable proof components, self-contained replay sources, and a staged integration proposal.

Contents

1 Recommendation and scope **2**

2 What is already simplified **2**

2.1 Use current source, not a historical checkpoint count 2

2.2 Do not repeat completed work 3

2.3 Preserve the exact object and proof boundary 3

3 Where the runtime repetition comes from **4**

4 A guarded frame theorem for the actual executor **4**

4.1 Symbolic configurations 4

4.2 Lift a finite body once, for all frames 5

4.3 Finish with the existing Home theorem 6

4.4 Independent Rocq check of the local frame rule 6

5 Connect runtime templates to exact source actions **7**

5.1 Reject prefix escape explicitly 7

5.2 The combined quantified theorem 8

5.3 Match the actual site gap 9

6 Use semantic coverage as the consumer interface **9**

6.1 Precise remaining assembly obligations 10

7 A second component: extensional source rewriting in Rocq **10**

7.1 The exact local change 10

7.2 Use pointwise register equality 11

7.3 Checked concrete result and its limitation 12

8 Validation and artifact interpretation **12**

9 Integration plan and meaningful measurements **13**

10 Further research questions **14**

Attribution **15**

1 Recommendation and scope

The recommended approach is *contextual proof reuse*: isolate a local transformation, state exactly what surrounding data it cannot inspect or alter, and prove the transformation once for every admissible surrounding context. For K215, this is useful in two different layers:

1. In the binary runtime proof, a short control routine changes a suffix of the encoded program counter. Its earlier prefix is preserved until the existing Home routine scans it and returns to dispatch. One quantified contract can therefore replace many repeated symbolic executions.
2. In the source proof, a node whose body increments two registers can be replaced by the opposite order. The surrounding program sees the same register values and the same structured exit. A generic contextual transport theorem therefore propagates two local proofs through all their occurrences.

These ideas were already identified in two repository proposals: `round13-left-frame` [5] and `k215-simplification-3` [6]. Their discovery is not claimed here. The contribution of this delivery is to supply missing universal proofs and checked concrete instances, including an extensional-register interface that preserves Rocq's closed assumption boundary.

The most valuable near-term integration is the 35-template runtime family. Its generic theorem, exact literal executions, right-tape normalization, and source control actions have now been checked. The remaining work is finite source-occurrence assembly, semantic coverage, and endpoint rechecking. The two-node source rewrite is a second, complementary component. It still needs a certified source-to-CFG connection before it can replace the current source correspondence certificate.

The final theorem remains

$$\mathrm{Halts}(K215, \mathrm{blank}) \quad \Longleftrightarrow \quad \text{ZFC has a finite derivation of falsity.}$$

Its negation yields the intended syntactic consistency equivalence. A proof of this equivalence does not assert that K215 fails to halt unconditionally. The repository distinguishes its external Lean/Rocq theorems from a separately encoded derivation in PA or in ZFC's own proof syntax [1]. This delivery preserves that distinction.

2 What is already simplified

2.1 Use current source, not a historical checkpoint count

The review is pinned to commit `cbf94cb552a3b49509e0eee80e7455d4eb5974c6`. The phase-15 checkpoint records an endpoint with 176 project modules and 2,052,142 selected source bytes [4]. An independent recursive import walk of the maintained `KEndpoint215` at the review pin instead finds **174 project modules, 1,752,650 raw source bytes, and 37,648 lines**, with `Init` as the only external import boundary. The maintained reproduction driver's inventory routine gives the same census.

This is a *static current-source measurement*. It is not a new compilation of those 174 modules, and historical phase-15 receipts must not be represented as fresh checks of all current bytes. The source census and its exact hashes are included in the archive. Later direct-seed/interface and

source-calculus changes explain why the maintained closure can differ from the checkpoint summary. No claim that the current endpoint is invalid follows from this documentation lag.

Scope	Modules	Source bytes	Interpretation
Phase-15 endpoint	176	2,052,142	Historical checkpoint and its receipts
Current import closure	174	1,752,650	Fresh static census at the review pin
New component checks	See the included build records		Fresh focused compilation; no full endpoint replay

Table 1: Different evidence scopes must not be conflated.

2.2 Do not repeat completed work

The repository already uses offset-bisimulation symmetry, normalized source correspondence, shared dominated-implication arguments, a generic AX word composition, shared enumerator contracts, smaller semantic import boundaries, shared recursion and quotient syntax, and a compact dense-machine invariant [3]. The source correspondence already has 1,426 normalized checkpoint entries, rather than the older 2,240. The current source also contains direct compilation of the enumerated calculus and the updated seed route. Recommending these as entirely new changes would miss the maintained baseline.

The final endpoint itself is already a short composition. Shortening its few lines further would have little effect on the cost of checking the repeated runtime and source certificates. We instead target statements that account for repeated semantic work.

2.3 Preserve the exact object and proof boundary

The accepted object has 215 working states, two symbols, a two-way infinite tape, blank input, initialization included, and a separate halt. The dense table has 430 transition rows [2, 1]. The following identities are retained:

Dense table SHA-256
`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`

Native table SHA-256
`9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2`

Source SHA-256
`17af207195179b2508bf91154cd00739374f8862207ca21b9a3cb9098aa6f5ba`

The fresh focused Lean proofs use ordinary kernel-checked `decide`, not `native_decide`, and contain no accepted `sorry` or added axioms. Their reports stay within the existing Lean allowance `propext`, `Classical.choice`, and `Quot.sound`. The fresh Rocq source component reports are closed under the global context. Conditional hypotheses such as the four existing register-routine contracts are explicit theorem parameters, not postulated axioms.

3 Where the runtime repetition comes from

The current source DAG has 202 distinct nodes and expands to 4,751 leaves. The independent audit follows the maintained 234-role decoder and compares all 430 formal table rows with the native table after the explicit label translation. The leaf and guarded-outcome counts are:

Primitive kind	Source leaves	Guarded outcomes
Increment	1,900	1,900
Decrement/test	1,425	2,850
Break	98	98
Continue	1,327	1,327
Halt	1	1
Total	4,751	6,176

Table 2: A decrement contributes separate zero and nonzero outcomes.

The framed family covers 1,326 continue occurrences and 96 break occurrences: 1,422 control cases in total. It uses exactly 35 distinct local bodies. The three control fallbacks are `001/CB1`, `1000111/CC3`, and `11011/CB1`. The 35 bodies contain 444 symbolic operations altogether, with a maximum of 39 in one body. All 35 bodies are used by the audited occurrence map.

The proposed expensive replay census is therefore

$$35 + 3 + 1900 + 2850 + 1 = 4789,$$

compared with 6,176 literal outcome replays. The difference is

$$6176 - 4789 = 1387, \qquad \frac{1387}{6176} \approx 22.4579\%.$$

All 6,176 exact source-outcome bindings remain necessary. A template instance still needs a route, suffix decomposition, prefix-scanner premise, action identity, and gap arithmetic. This count does not measure total proof size, object size, memory, or wall-clock time. It also does not justify removing any outcome from source coverage.

Native state labels are not the maintained formal labels. In particular, native 301, 302, and halt 254 become formal 102, 120, and 230. State 302 occurs in the actual local traces, so this translation cannot be silently omitted. The new Lean finite equations are evaluated against the maintained `KTable215.tm215`; the independent audit also checks the complete row translation.

4 A guarded frame theorem for the actual executor

4.1 Symbolic configurations

The maintained executor describes a symbolic configuration by

$$s = (q, o, L, R),$$

where q is a state, o indicates which list carries the head, and L, R are lists of symbolic segments. Segments are a single Boolean cell, a constant run of length $an + c$, or a register-file box. The

interpretation $\mathrm{conc}_{n,v}(s)$ is the complete canonical zipper configuration for gap parameter n and register list v.

For any segment list F, define

$$\mathrm{frame}_F(q, o, L, R) = (q, o, L +\!\!+ F, R).$$

This appends an untouched left context. It does not trim or normalize the left list. Define the guard

$$\mathrm{safe}(q, o, L, R) \quad \Longleftrightarrow \quad \neg(o = \mathrm{true} \ \wedge \ L = []).$$

Theorem 4.1 (Guarded append-left naturality)**.** *For every table M, branch condition b, segment list F, and symbolic configuration s, if s is safe, then*

$$\mathrm{sstep}_{M,b}(\mathrm{frame}_F(s)) = \mathrm{map}\big((t, m) \mapsto (\mathrm{frame}_F(t), m),\ \mathrm{sstep}_{M,b}(s)\big).$$

The equation preserves both successful outputs and failure, and preserves the executor's progress Boolean.

Proof. Destructure the head side and the leading segment. On a right-side bit, a machine step either prepends the written bit to L, leaves L unchanged, or fails. All three operations commute with appending F. A left-side bit reorientation removes the first bit from a nonempty L; its tail becomes $(\mathrm{tail}\, L) +\!\!+ F$. For a constant run, the whole-run self-loop branch prepends or transfers the same run, and the two run-peeling helpers commute with append by inspection of their natural-number cases.

The register-box branch leaves the left list unchanged or prepends one fixed bit. The oracle result is independent of F. Unsupported symbolic shapes remain unsupported. The only other branch is a left-facing empty list; it is excluded by the guard. These are all branches of the maintained `sstep`. The checked Lean theorem is `KLeftFrame.sstep_natural`. □

Remark 4.2 (Why the guard is essential)**.** For $s = (q, \mathrm{true}, [], R)$, the original executor inserts an implicit blank when reorienting. After appending $F = [\mathrm{bit}(1)]$, it instead reads that explicit one. Hence the unguarded equation is false, even for a table with no machine transitions. The package contains a checked Lean counterexample. The guard must hold before *every* local step, not only at the initial configuration.

4.2 Lift a finite body once, for all frames

Define a bounded local replay by

$$\mathrm{runLocal}_{M,b}(0, s) = \mathrm{Some}(s),$$
$$\mathrm{runLocal}_{M,b}(j+1, s) = \begin{cases} \mathrm{runLocal}_{M,b}(j, t), & \text{if } s \text{ is safe and } \mathrm{sstep}_{M,b}(s) = \mathrm{Some}(t, m), \\ \mathrm{None}, & \text{otherwise.} \end{cases}$$

It is permitted to *end* with an empty left list. It cannot execute the next, unsafe boundary reorientation.

Theorem 4.3 (Local replay under every appended frame)**.** *Assume the existing register-routine hypotheses `BoxLemmas M`. If* $\mathrm{runLocal}_{M,b}(j, s) = \mathrm{Some}(t)$*, then, for every F, n, v satisfying the existing branch-validity predicate,*

$$\mathrm{Reach}_M\big(\mathrm{conc}_{n,v}(\mathrm{frame}_F(s)), \mathrm{conc}_{n,v}(\mathrm{frame}_F(t))\big).$$

Proof. Induct on j. At zero the successful equation identifies the endpoints, so reflexivity applies. At a successor, successful replay exposes a safe configuration, a successful symbolic step, and a shorter replay. Apply Theorem 4.1; apply the existing `sstep_sound` theorem to that framed step; then compose its concrete reachability with the induction hypothesis. No additional tape interpretation or trusted evaluator is used [11]. □

The conclusion is reflexive-transitive reachability, not positive reachability. Some symbolic operations are reorientations or zero-length run normalizations. Requiring positive progress at every symbolic step would be incorrect. The existing final Home theorem provides the necessary positive machine progress for the whole source action.

4.3 Finish with the existing Home theorem

All 35 local bodies end at $(2, \text{true}, [], R')$. Choose

$$F = \text{map}(\text{bit}, \text{rev}(P)) +\!\!+ [\text{bit}(1)].$$

The caller supplies the existing scanner premise

$$\texttt{scanBound}(0, \text{map}(\text{bit}, \text{rev}(P))) = \text{Some}(k).$$

The already proved `scanBound_sound` gives the left-word scan across $\text{rev}(P)$. The final one-cell marker resets its counter. Applying the existing `home_word` then returns to state 84 with left marker [1] and right word $P +\!\!+ \text{flat}_{n,v}(R')$ [12]. Thus:

Corollary 4.4 (Positive local-plus-Home contract)**.** *Assume `BoxLemmas M`, `KHome.Rows M`, and `KHome.Boundary M`. If* $\text{runLocal}_{M,b}(j, s) = \text{Some}(2, \text{true}, [], R')$*, then for every accepted prefix P, gap n, and valid register list v,*

$$\text{Reach1}_M\left(\text{conc}_{n,v}(\text{frame}_F(s)),\ z_c(84, [1], P +\!\!+ \text{flat}_{n,v}(R'))\right).$$

Proof. Compose Theorem 4.3 with the positive Home execution using `Reach.trans_reach1`. The explicit empty local left list makes the Home entry exactly the caller's frame; no new theorem about crossing an opaque boundary is required. □

There is no condition that P end in one. In the actual occurrence map, 721 accepted prefixes end in zero. Adding an ends-in-one premise would needlessly lose sharing. There *is* a blank far-left exterior premise inherited from the zipper/Home boundary semantics. The generic algebraic frame theorem is valid for arbitrary segment frames, but that does not turn Home into an arbitrary-nonblank-exterior theorem.

4.4 Independent Rocq check of the local frame rule

The archive also ports the guarded rule to the maintained Rocq `Runtime230Core` semantics. The effective Rocq table agrees with Lean on all 430 actions after the documented native-label translation. The same 35 literal templates are checked independently with `vm_compute`: 444 symbolic steps in total, with a maximum local length of 39.

`Refactor.LeftFrame` proves the append-frame naturality theorem, its essential-guard counterexample, and finite local replay soundness under the four inherited box-routine hypotheses.

The template module `Refactor.LeftFrame215` proves `all_checked` and a universally quantified `template_replay`: every listed local replay remains valid for every symbolic left frame. Composing it with the generic soundness theorem gives the corresponding concrete reachability contract. The four generic and two concrete assumption reports are closed under the global context.

Nine exact source files, including the guarded frame core, were freshly compiled with Coq 8.18.0 and recursively checked with `coqchk`. The tenth source, the generated template client, was then freshly compiled against those exact dependency objects. Its receipt is separate: a second recursive sweep including that client was not run. The included replay driver can freshly compile and recursively check the complete ten-source component.

This Rocq result has a narrower scope than the Lean result below. It proves the local replay and framed reachability, but does not yet port the Home scan or the exact source-action adapter. It is therefore not a claim of 35 positive complete Rocq control contracts or of a rebuilt Rocq endpoint.

5 Connect runtime templates to exact source actions

5.1 Reject prefix escape explicitly

The original path arithmetic uses least-significant-first operations internally. For example, `stripRev b k` removes cells through the kth occurrence of bit b, and `incRev` performs fixed-width increment. Their original definitions remain unchanged.

The new `KControlFrame` module supplies two option-valued sufficient checks. `stripChecked` fails if the specified suffix does not contain enough matching bits. `incChecked` fails if a carry would leave the suffix. Successful values certify exact arithmetic under every added prefix.

Lemma 5.1 (Safe stripping and incrementing)**.** *If* $\mathit{stripChecked}(b, k, s) = \mathrm{Some}(t)$*, then for every list* p*,*

$$\mathit{stripRev}(b, k, s +\!\!+ p) = t +\!\!+ p.$$

If $\mathit{incChecked}(s) = \mathrm{Some}(t)$*, then*

$$\mathit{incRev}(s +\!\!+ p) = t +\!\!+ p.$$

Proof. For stripping, induct on s, generalizing the number of remaining matching bits. At zero remaining bits the list is unchanged. Otherwise a matching head decreases the counter and a nonmatching head does not; successful evaluation ensures recursion never reaches an empty list with a positive counter. For increment, a false head ends the carry and a true head propagates it. Successful evaluation excludes propagation past the end. Induction gives both equations. Reversal yields the corresponding most-significant-first suffix statements. □

Let `controlSuffix` run the stripping check for continue, and stripping followed by checked increment for break. It rejects the other edge kinds. The checked generic theorem is

$$\texttt{controlSuffix}(e, S) = \mathrm{Some}(E) \implies \forall P, R, \texttt{execute}(e, P +\!\!+ S, R) = \mathrm{Some}(P +\!\!+ E, R).$$

It is not sufficient merely to observe equality for one concrete caller prefix. In a break instruction, both the stripping and the subsequent carry must stay inside the chosen suffix.

5.2 The combined quantified theorem

A descriptor records a runtime-template index, a numerical entry state q, a nonempty suffix S, a result suffix E, and its source edge e. A finite kernel check verifies all of the following for each of the 35 descriptors:

- the descriptor selects the intended literal runtime template;
- its entry is the suffix port associated with q and S;
- its control action passes `controlSuffix`;
- $|E| \le |S|$, and its right endpoint has exactly the required normalized suffix, gap, marker, and unchanged register box;
- the descriptors enumerate all template indices $0, \dots, 34$.

Write $\mathcal{R}(v) = \texttt{sepsR}(v)$, with concatenation of Boolean words written by juxtaposition. For each admitted descriptor, define

$$A(q, P, S, n, v) = z_c\big(q, \mathrm{rev}(\mathrm{init}\, S)\,\mathrm{rev}(P)1, \mathrm{last}(S)0^{n+16}1\mathcal{R}(v)\big),$$
$$B(P, S, E, n, v) = z_c\big(84, 1, PE0^{n+16+|S|-|E|}1\mathcal{R}(v)\big).$$

Theorem 5.2 (All 35 complete control contracts)**.** *Assume the existing* `BoxLemmas`*, Home rows and Home boundary contracts for the maintained* `tm215`*. For every one of the 35 descriptors, every prefix P accepted by the existing scanner, every $n \in \mathbb{N}$, every register list v with $|v| \ge 10$, and every source register function R,*

$$\mathrm{Reach1}_{\texttt{tm215}}(A(q, P, S, n, v), B(P, S, E, n, v))$$

and

$$\texttt{execute}(e, PS, R) = \mathrm{Some}(PE, R).$$

Proof. Extract the literal guarded replay from the checked template list and apply Corollary 4.4. The alignment check identifies the initial symbolic configuration. Its right-normal-form equality, transported through the existing `flatT_norm`, identifies the final *whole* right tape for every n, v. The checked length inequality justifies natural-number gap arithmetic. Apply the source-action lemma to the checked `controlSuffix` equation. The descriptor-index check ensures that this supplies every template, not merely a selected example. The actual checked theorem is `KFramedControl215.control_contract`. □

Only the right endpoint is normalized. One must not use `normL` to trim trailing blanks before appending the frame. For example, [0] and [] can represent the same blank unframed exterior, while appending [1] gives $[0, 1]$ and [1], which place the marker at different distances. The frame proof deliberately preserves the complete symbolic left fragment.

As an example, the descriptor with entry state 13 and control instruction `CC2` has $S = 111$ and $E = 1$. Its local body takes eight symbolic steps. For every allowed prefix P, continue twice takes $P111$ to $P1$, leaves registers unchanged, and increases the represented gap from $n + 16$ to $n + 18$. Choosing a prefix ending in zero is permitted. The theorem is independent of that example and quantifies over all admissible prefixes.

5.3 Match the actual site gap

The maintained site format uses gap $a + (64 - |Q|)$, where the consumed source word Q has length at most 48. If $Q = PS$, instantiate the template parameter with

$$n = a + (48 - |Q|).$$

Then $n + 16 = a + (64 - |Q|)$. Since $|E| \leq |S|$, the output gap equals

$$n + 16 + |S| - |E| = a + (64 - |PE|).$$

These are equalities in natural-number arithmetic; the length bounds are essential when simplifying subtraction. The template's source action also has no prefix carry or marker overflow. The independent occurrence audit checks the exact action, route, decomposition, scanner condition, and gap delta at all 1,422 assigned sites.

6 Use semantic coverage as the consumer interface

The maintained `KSim.Covered` currently asks that each decoded leaf satisfy the bounded Boolean `checkEdge`. The generic endpoint consumes that exact predicate [9, 10]. Theorem 5.2 is a semantic reachability theorem. It does not imply that the old executor, with its fixed fuel, returns `true`. A completeness theorem of that kind is neither available nor generally valid for arbitrary bounded execution.

Consequently, the new theorem cannot simply be substituted for `KSites215.covered`. The correct interface is an edge contract:

$$\mathrm{EdgeContract}(M, Q, e),$$

which requires the same path/register bounds and the appropriate positive whole-tape executions, with separate zero and nonzero decrement branches and a separate halt clause. The already developed `semantic-coverage` proposal supplies a semantic consumer, a sound adapter from `checkEdge`, and mixed coverage [8]. Reuse and rebase that work; do not reimplement the simulation induction independently for each template family.

The intended proof has two producers of the same contract:

1. *Literal producer.* The existing checked executor proves the remaining 4,754 outcomes: 1,900 increments, 2,850 decrement branches, three control fallbacks, and one halt.
2. *Framed producer.* The decoder-prefix theorem reaches the exact primitive port; Theorem 5.2 performs the local control action and positive Home return. Exact finite binding data instantiate it at the other 1,422 outcomes.

The consumer proves total source simulation from these contracts. A forward step must have positive machine duration, so a halting machine cannot simulate infinitely many source steps in zero machine time. The unchanged Home theorem supplies this positivity for the new control family. The halt clause continues to identify an actual terminal state, rather than an exhausted symbolic evaluator.

6.1 Precise remaining assembly obligations

The following are still required before claiming that the shorter proof has replaced the production endpoint:

1. Prove in the kernel that each selected source occurrence has its exact consumed path, source action, and decoder route. Numerical equality of an entry state is not itself a `RoutedEntry` proof: different decoder roles can share a physical state.
2. Assemble the 1,422 template instances, using the positive concrete decoder prefix and the exact input/output shape and gap rewrites. The new generic source-action theorem removes the need to reprove prefix preservation at every instance.
3. Retain the three exceptional control cases and all other outcomes through the existing sound checker. Establish total semantic coverage of all 4,751 leaves and 6,176 outcomes, including initial entry and the literal halt leaf.
4. Add or rebase a semantic endpoint adapter while preserving the original endpoint type and compatibility clients. Old Boolean coverage should remain a sufficient way to use the generic endpoint.
5. Rebuild affected dependencies and the exact endpoint clients using the production versions, inspect assumption reports, and separately measure source size, generated data, object size, time, and memory.

The new package does not include a completed full-site formal assembly or a fresh ZFC endpoint build. Its independent Python audit supplies exact finite data validation, not a substitute for these kernel assembly obligations. The earlier round13 prose mentions an all-leaf residual packet under another proposal path; this review does not treat that historical claim as a fresh binding of sources that are absent at the review pin.

7 A second component: extensional source rewriting in Rocq

7.1 The exact local change

The earlier CFG investigation found that the source programs differ in the order of increment-only pairs at two shared `dagPerm` nodes [6]:

node	original body	commuted body
188	(Inc 6, Inc 7)	(Inc 7, Inc 6)
190	(Inc 6, Inc 1)	(Inc 1, Inc 6).

The proposed program `dagPermComm` changes exactly these two node definitions; its root and node count are unchanged. It is an intermediate source presentation, not a new binary machine.

Two source templates account for 256 expanded swaps in the prior structural analysis. This is precisely where shared-DAG reasoning helps: the semantic proof should depend on the two changed node bodies, not on every expanded occurrence. The existing `LocalProgram` proposal has a contextual transfer principle with literal register-function equality [7]. That equality is stronger than needed for these reordered updates.

7.2 Use pointwise register equality

A register file in the Rocq semantics is a function $R : \mathbb{N} \to \mathbb{N}$ [13]. Define

$$R \simeq S \quad \Longleftrightarrow \quad \forall i,\ R(i) = S(i).$$

Pointwise equality lets us prove update commutation without converting this statement into equality of function objects. That conversion would ordinarily invoke functional extensionality in this proof style. The delivered source transport never needs it.

Lemma 7.1 (Increment commutation)**.** *Let I_a increment register a. For all a, b, R,*

$$I_b(I_a(R)) \simeq I_a(I_b(R)).$$

The lemma even permits $a = b$.

Proof. If $a = b$, both sides are the same nested increment. Otherwise fix an index i. If $i = a$, both sides have value $R(a) + 1$; if $i = b$, both have value $R(b) + 1$; otherwise both have $R(i)$. This is an indexwise argument using only decidable equality of natural numbers. No function-equality axiom occurs. □

The actual structured semantics uses judgments

$$\mathrm{run}_p(e, R, x, R'),$$

where x is the normal, break, continue, or halt exit (with the existing depth parameters where applicable). The new one-direction node contract is

$$\begin{aligned}\mathrm{NodeTransport}(p, q, n) \ :\Longleftrightarrow\ & \forall R, x, R',\ \mathrm{run}_p(\mathrm{Node}\, n, R, x, R') \\ & \Longrightarrow \forall S,\ R \simeq S \Longrightarrow \exists S', \mathrm{run}_q(\mathrm{Node}\, n, S, x, S') \wedge R' \simeq S'.\end{aligned}$$

The structured exit is preserved exactly. Only the register representation is related pointwise.

Definition 7.2. `ClosedExt p q` says that for every node n, either its two child edges are literally unchanged, or `NodeTransport p q n` holds.

Theorem 7.3 (Extensional contextual transport)**.** *If `ClosedExt p q`, then every finite structured run of p transports to a run of q with the same exit and pointwise related output registers, starting from any pointwise related input registers.*

Proof. Induct on the derivation of the finite structured run. Primitive increments preserve pointwise equality under update; decrement branches agree because the tested register values agree. Primitive break, continue, and halt retain the same registers and exit.

At a node, inspect the closure condition. For a changed node, use its supplied whole-node transport contract. For an unchanged node, invoke the induction hypotheses on the first child, second child, and any loop-restart derivation required by the actual run constructor. Pass the intermediate pointwise register relation to the next induction hypothesis. Rebuild the same node constructor using equality of the child-edge pair.

This handles all structured exit constructors, including the continuation case that restarts the node. There is no acyclicity or node-rank hypothesis: the induction is on the finite execution evidence, not on the source graph. □

Corollary 7.4 (Halting equivalence)**.** *If* `ClosedExt p q`*,* `ClosedExt q p`*, and the root edges agree, then for every initial register file R,*

$$\text{iter_halts}(p, R) \iff \text{iter_halts}(q, R).$$

Proof. Transport each finite root run with Theorem 7.3. For a normal root exit, feed the pointwise output relation into the induction hypothesis for the remaining iterations; for a halt exit, finish. Apply this argument in both directions, starting with $R \simeq R$. □

7.3 Checked concrete result and its limitation

Both changed nodes contain exactly two increments. Their structured runs must exit normally with the corresponding pair of updates. The increment lemma proves the node contract in either direction. All other nodes are literally unchanged. The checked result is

$$\forall R, \quad \texttt{iter_halts dagPerm R} \iff \texttt{iter_halts dagPermComm R}.$$

The package also supplies transport of arbitrary structured edge runs in both directions, preserving every exit and pointwise output registers.

The generic and concrete components were freshly compiled with Coq 8.18.0, using the actual pinned `RM.v` and current `DagPerm.v`. All six printed assumption reports are closed under the global context. Recursive `coqchk` of the concrete result passes. This is a focused compatibility check, not a claim that the full production Rocq 9.1.0 endpoint has been recompiled with Coq 8.18.0.

What this does not yet prove. The previous Python experiment reports ordered-CFG equality of `p215` with the two-node-commuted source. The new Rocq theorem does not turn that experiment into a kernel theorem. In particular, it does not eliminate the 1,426-entry normalized source correspondence certificate by itself.

To complete that route, certify flattening or a canonical CFG checker that preserves initial entry, every decrement branch, explicit halt, and all eliminated control transfers. Pure-control cycles must be rejected or represented as divergence. Increment-only cycles must not be replaced by a finite increment vector. A generic certificate about finite elimination traces or a decreasing rank can establish that normalization does not hide infinite zero-progress execution. The earlier analysis already identifies these obligations; the present result closes the local source-commutation part while leaving flattening explicit.

8 Validation and artifact interpretation

The archive separates three kinds of evidence:

Fresh kernel checks. The Lean frame rule, guarded replay, Home composition, source-action framing, all 35 literal runtime checks, descriptor alignment, and combined universal control contracts. The Rocq pointwise contextual transport and exact two-node source rewrite, with recursive checking; also the separately checked Rocq local frame rule and 35 local replays, with the precise recursive-check boundary stated above.

Fresh finite audits. An independently authored Python checker binds the 35 templates and 1,422 occurrences to the pinned machine, maintained decoder, and complete source expansion.

It checks all 444 local steps, full state-label translation, source-action guards, right-tape arithmetic, and the exact three fallbacks. It contains explicit negative examples at the unsafe boundary.

Historical context. The repository's endpoint/checkpoint receipts and earlier proposal findings. These explain what was already accomplished and where the new components fit. They are not relabelled as this delivery's fresh endpoint checks.

The source tree contains the unchanged focused dependencies required for offline component replay. Provenance manifests distinguish those inherited files from new modules and bind each to exact bytes. Compiler binaries and compiled proof objects are not shipped as substitutes for source checking. Raw successful logs, exact compiler versions, commands, hashes and results are retained. Failed development attempts, where retained, are separate from the accepted fresh replay; a printed `sorryAx` in a failed attempt is not an accepted theorem report.

The principal new Lean declarations are:

- `KLeftFrame.sstep_natural`, `runLocal_sound`, and `local_home`;
- `KControlFrame.controlSuffix_frame`;
- `KLeftFrame215.all_checked` and `template_home`;
- `KFramedControl215.all_aligned`, `descriptor_indices`, and `control_contract`.

For Rocq, `run_transfer_ext` and `iter_iff_ext` supply generic transport. The declarations `swap_increment_node` and `dagPerm_comm_iter_iff` supply the concrete application. The runtime component adds the guarded frame and local replay declarations described in Section 4.

See the archive's `README.md` for exact replay commands and the final machine-readable verification summary for the accepted compilation scope. The integrity verifier checks archive bytes; the proof compilers and recursive checker establish the corresponding formal claims. The inventory and finite audit scripts do not claim to establish an internal ZFC derivation.

9 Integration plan and meaningful measurements

Stage	Change	Completion criterion
1	Import the new frame and control-action lemmas	Fresh focused checks on the pinned production toolchains; existing executor definitions unchanged
2	Rebase semantic coverage with compatibility adapters	Old literal checker still produces the same contracts; old endpoint API clients still typecheck
3	Assemble 1,422 exact template occurrences	Kernel-bound decoder roles, paths, source actions, gap equations, and prefix premises; no missing outcomes
4	Compose mixed semantic coverage	All 4,751 leaves and 6,176 outcomes covered, with three control fallbacks and a separate terminal case

Stage	Change	Completion criterion
5	Rebuild the exact K215 endpoint	Same literal-table hashes, same final theorem types, allowed assumption reports, no incomplete providers
6	Develop certified source flattening	Compose with the checked two-node Rocq equivalence; preserve initial entry, branch order, halt and divergence

For stages 3–5, benchmark the baseline and candidate on the same toolchain, machine, resource limits, and fresh or equivalently warmed object conditions. Report at least:

1. maintained handwritten source and generated source separately;
2. serialized finite data and compiled object sizes;
3. elaboration/kernel-check time and peak memory for the runtime family;
4. affected endpoint build time, with dependency reuse explicitly identified;
5. number of independent expensive executions and number of total bindings.

The runtime contract module adds generic proof infrastructure. Its first adoption may increase source lines even while reducing repeated computation. Do not report the 22.46% obligation reduction as a 22.46% source reduction or time reduction. Similarly, a small final wrapper does not establish that the underlying checked dependency closure is smaller. The delivered components make a concrete comparison possible; they do not predetermine its result.

For Rocq, pointwise transport is useful independently of the two-node normalizer. It permits future register-permuting or update-reordering local contracts without importing functional extensionality. The exact current closed endpoint should remain the reference assumption profile.

10 Further research questions

1. **Can the canonical CFG be computed and justified from the compact source DAG?** The two-node commutation theorem now supplies the local rewrite. A certified extraction theorem could remove separately maintained path-pair correspondence data. It must explicitly handle control-only and increment-only cycles, malformed walks, initial entry, and the true halt leaf.
2. **Can the same frame principle cover increments and decrements?** They account for 4,750 guarded outcomes, substantially more than the control family. The likely abstraction combines unchanged prefix with the existing register effect and a parametric gap update. Decrement's two branch premises and the extra spare block produced by increment must remain explicit.
3. **Can a small checker discover the minimal protected suffix?** A reflected footprint analysis could find the first boundary that a local trace must not cross. Its correctness should imply the guarded frame rule, and its rejection path should fall back to the original checker. This would reduce hand-maintained occurrence assignments without trusting the discovery algorithm.

4. **What is the best compiled representation of the 35 descriptors and 1,422 bindings?** A compact indexed array, source-DAG annotation, or shared proof tree may reduce evaluation and source maintenance differently. Compare actual object size and checked execution cost; printed line count alone is a poor guide.
5. **How much of the symbolic-progress flag can remain local?** The present proof needs no positivity from the guarded fragment because Home supplies it. More generally, require progress at a designated observable boundary instead of rebuilding positive duration bookkeeping for each administrative symbolic operation.
6. **Can extensional contextual transport be shared across both proof lanes as a specification?** The languages should retain independent small proofs, but agree on the exact exit relation, pointwise register relation, node replacements, and certificate data. This avoids a new trusted cross-language translator.
7. **What is required for the separate internal ZFC endpoint?** The existing proof-assistant endpoint and the reviewed ordinary mathematical argument are distinct from an encoded derivation in ZFC syntax. Any future internalization should expose its proof-code translation and arithmetic formalization instead of treating the external theorem as that object.

Attribution

The 35-template discovery and empty-left decomposition come from the existing Round 13 left-frame proposal. The two-node source comparison comes from the third K215 simplification proposal. The contextual source-transfer design builds on the `common-macros` proposal and the existing Wade register-machine semantics. The symbolic executor, zipper model, Home routine, source calibration, K215 construction, and original Lean/Rocq proofs are inherited from the named contributors and upstream projects documented in ProveIt. New work here comprises the checked frame completion, source-action framing, 35 combined Lean contracts, independently checked Rocq local frames and replays, pointwise Rocq transport, concrete commutation proof, independent audits, and this integration analysis. Original notices and licenses accompany the reused source files.

References

[1] ProveIt contributors. Formal correctness of the exact 215-state machine. Review snapshot specified in Section 2.

[2] ProveIt contributors. K215: 215 working states for ZFC consistency.

[3] ProveIt contributors. Shorter checked K215 proofs.

[4] ProveIt contributors. Fifteenth K215 proof-simplification checkpoint.

[5] ProveIt proposal archive. Round 13 runtime: factor at the first empty-left Home boundary. Includes the original uncompiled contract sketch and template data.

[6] ProveIt proposal archive. Two local commutations instead of a general source certificate.

[7] ProveIt proposal archive. LocalProgram: whole-node source transfer.

[8] ProveIt proposal archive. K215: semantic coverage and a shared implication core.

[9] ProveIt contributors. Maintained KSim: decoder sites and register-machine simulation.

[10] ProveIt contributors. Maintained KEndpoint: generic endpoint composition.

[11] ProveIt contributors, adapting the K230 and m254 runtime proofs. KRuntimeCore: symbolic execution and soundness.

[12] ProveIt contributors. KHome: shared all-gap Home scan.

[13] K230 and upstream contributors. Pinned Wade register-machine semantics, RM.v. The archive retains the exact inherited source and attribution.